# 44. Reliability: Timeouts, Circuit Breakers, Graceful Shutdown

## 44. Reliability: Timeouts, Circuit Breakers, Graceful Shutdown

**Phase:** Phase 6 — Systems and infrastructure
**Estimated time:** 2 days
**Slug:** `44-reliability`

### Learning objectives

- Propagate deadlines everywhere
- Implement circuit breaker behavior
- Shut down HTTP servers gracefully
- Define SLOs mentally

### Prerequisites

Complete prior numbered lessons. Type every example yourself.

### Timeouts

Every external call needs a deadline. Inherit from request context when serving HTTP.

### Circuit breaker

When dependency error rate spikes, fail fast temporarily to allow recovery — then probe half-open.

### Graceful shutdown

```
go func() { _ = srv.ListenAndServe() }()
<-stop
ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
defer cancel()
_ = srv.Shutdown(ctx)
```

Drain in-flight requests; stop accepting new ones.

### SLO mindset

Availability and latency targets drive timeout budgets and retry policies.

### Common mistakes (especially from Python instincts)

- Kill –9 as the only deploy strategy
- No timeout on outbound calls
- Circuit breaker without metrics

## Exercises

Work in `exercises/44-reliability/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

**Do not open `solutions/44-reliability/` until you have an honest failing attempt.**

## Self-check

1. What does Shutdown do?
2. Fail-fast purpose?
3. Timeout budget meaning?

## Further reading

- https://pkg.go.dev/net/http#Server.Shutdown